Assignment Report

for

CS-3006: PARALLEL & DISTRIBUTED COMPUTING

Assignment 1

Ali Hamza Azam, I222126

February 1, 2025

Introduction

This report details the design and implementation of a parallel file analyzer in C++ for the Parallel and Distributed Computing Assignment 1. The analyzer efficiently processes large files by dividing the files into chunks and assigning them to threads. The implementation leverages a combination of multi-threading techniques using the POSIX Threads (pthreads) library and memory mapping to optimize file I/O operations.

Hardware Specifications

Chip:	Apple M3 Pro
Total Number of Cores:	11 (5 performance and 6 efficiency)
Memory:	18 GB
Threads:	11

Assignment Implementation Summary

Completed Tasks:

- **Task 1:** Fully implemented as specified, meeting all requirements.
- **Tasks 2 & 3:** Merged into a single implementation due to their overlapping logic and structural similarities. Both functionalities are operational and adhere to the assignment criteria.
- **Task 4:** Implemented in full, including error-handling logic. However, the provided dataset did not contain any "ERROR" entries, preventing practical validation of this component.

Program Execution and Parallel Processing Strategy

Main Function Overview

The program follows a three-phase workflow to process file data efficiently:

1. **Pre-processing:**
   - **File Mapping:** The entire input file is mapped into the process's virtual memory using the mmap_file function, enabling zero-copy access to the file contents.
   - **Workload Division:** The file is divided into chunks using the calculate_chunk_offsets_mapped function. This step ensures that each thread receives a complete set of lines without any partial or split rows.

2. **Main Processing:**
   - **Parallel Computation:** Multiple threads are spawned to process their assigned file chunks concurrently. Each thread computes local statistics, such as sum, maximum, minimum, count, and per-row/column aggregates.

3. **Post-processing:**
   - **Result Aggregation:** Once all threads complete their execution, the main thread aggregates the individual results into a global summary, which is then output.

Memory Mapping (mmap) Implementation

The mmap_file helper function maps the entire file into the process's virtual memory, providing several advantages:

- **Zero-copy access:** The file is mapped directly into memory, eliminating explicit data copying.
- **Direct pointer access:** The program can access file contents via pointers, simplifying parsing and enhancing performance.
- **Efficient I/O:** The operating system handles paging and caching efficiently, especially for large files.
- **Automatic resource management:** The function ensures proper file descriptor closure and memory unmapping during cleanup.

Chunk Division Strategy

Efficient parallel processing requires each thread to process complete, self-contained file portions. The calculate_chunk_offsets_mapped function achieves this by:

- Dividing the file into N equal segments (N = number of threads) based on the file size.
- Aligning chunk boundaries with the end of a line by adjusting nominal offsets until a newline character ('\n') is found.
- Preventing partial-line processing by ensuring each thread's chunk starts at a row boundary.

Results Calculation Using the Bash Script

The “run.sh” Bash script takes the results of the “compile.sh” Bash script and automates the experimental evaluation of the executables with various thread counts and core affinity settings. It iterates over predefined thread numbers (1 to 32) and affinity options (true and false), executing each variant multiple times (five repetitions per configuration). For each run, it clears system caches, invokes the executable, measures execution time, aggregates timings, logs results in CSV format, and captures error messages. This script provides a systematic way to evaluate how different parallel configurations impact the application’s runtime, supporting informed decisions about tuning the parallel processing strategy.

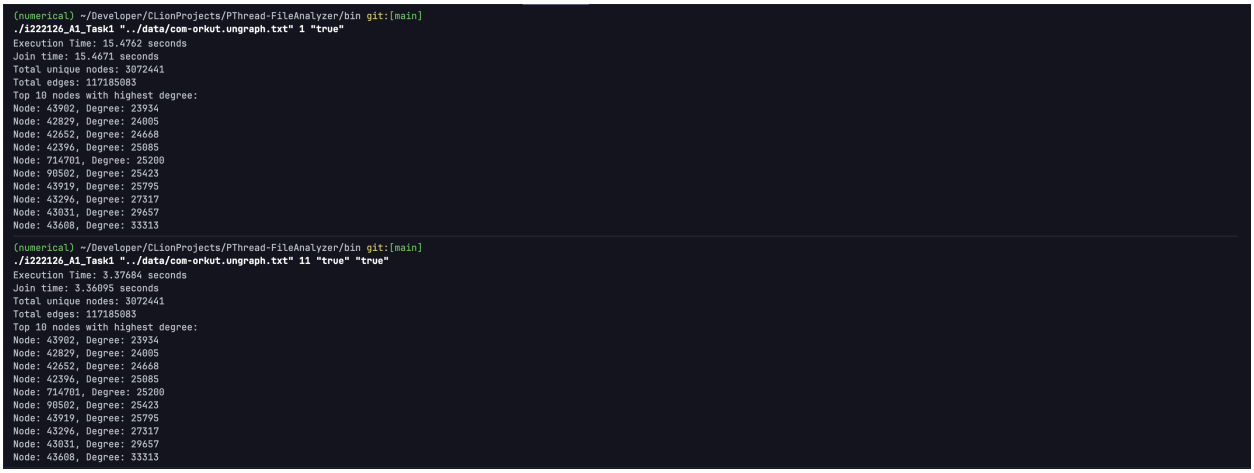
The “utilization.sh” Bash script is used to measure CPU trace for a 10-thread configuration for all tasks with and without thread affinity to observe any effect setting affinity might have on CPU utilization.

Task 1 : Graph File Analysis

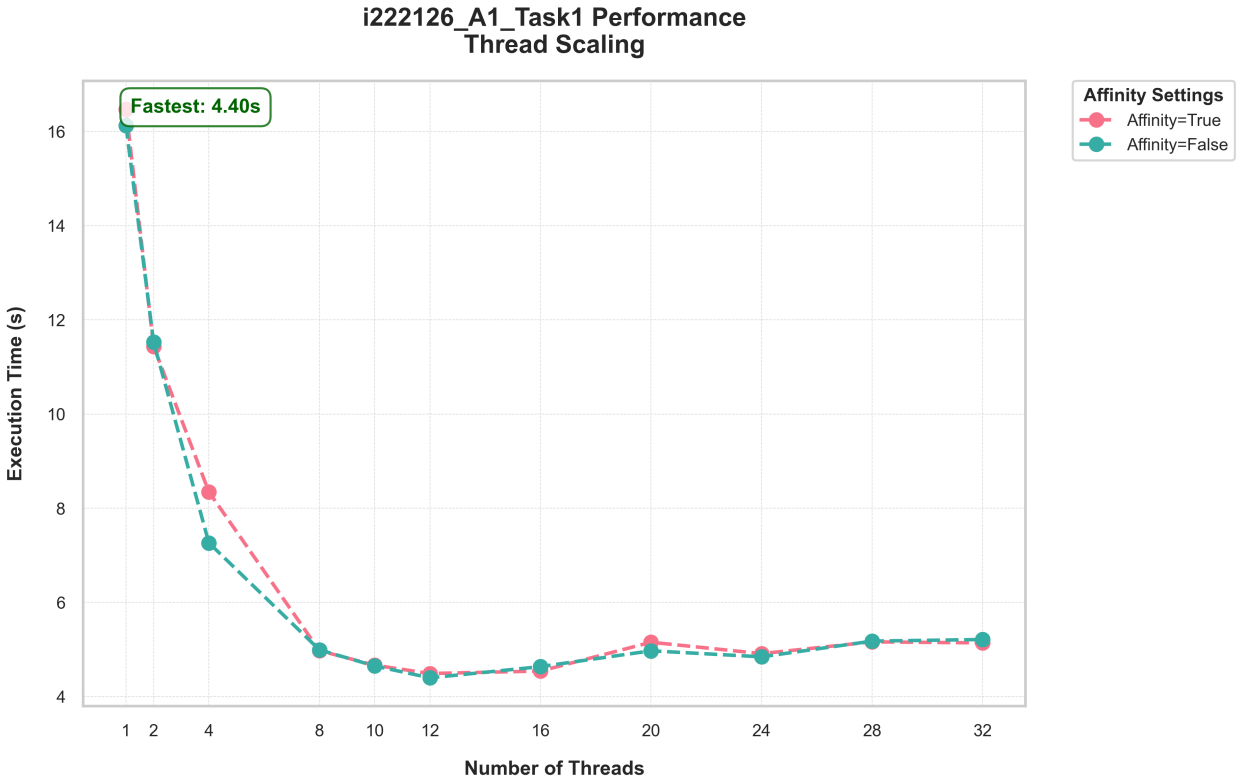
Approach

After memory-mapping the file and calculating chunk offsets, the main function assigns chunks to individual threads. Each thread processes its chunk independently, parsing lines to extract node pairs, count edges, and update local statistics. Once all threads complete, the main function aggregates results, collates node degree counts and total edge counts, and identifies the top 10 nodes with the highest degrees using a min-heap. This approach ensures even workload distribution and parallel processing, while the main thread consolidates results for output.

Sample Output



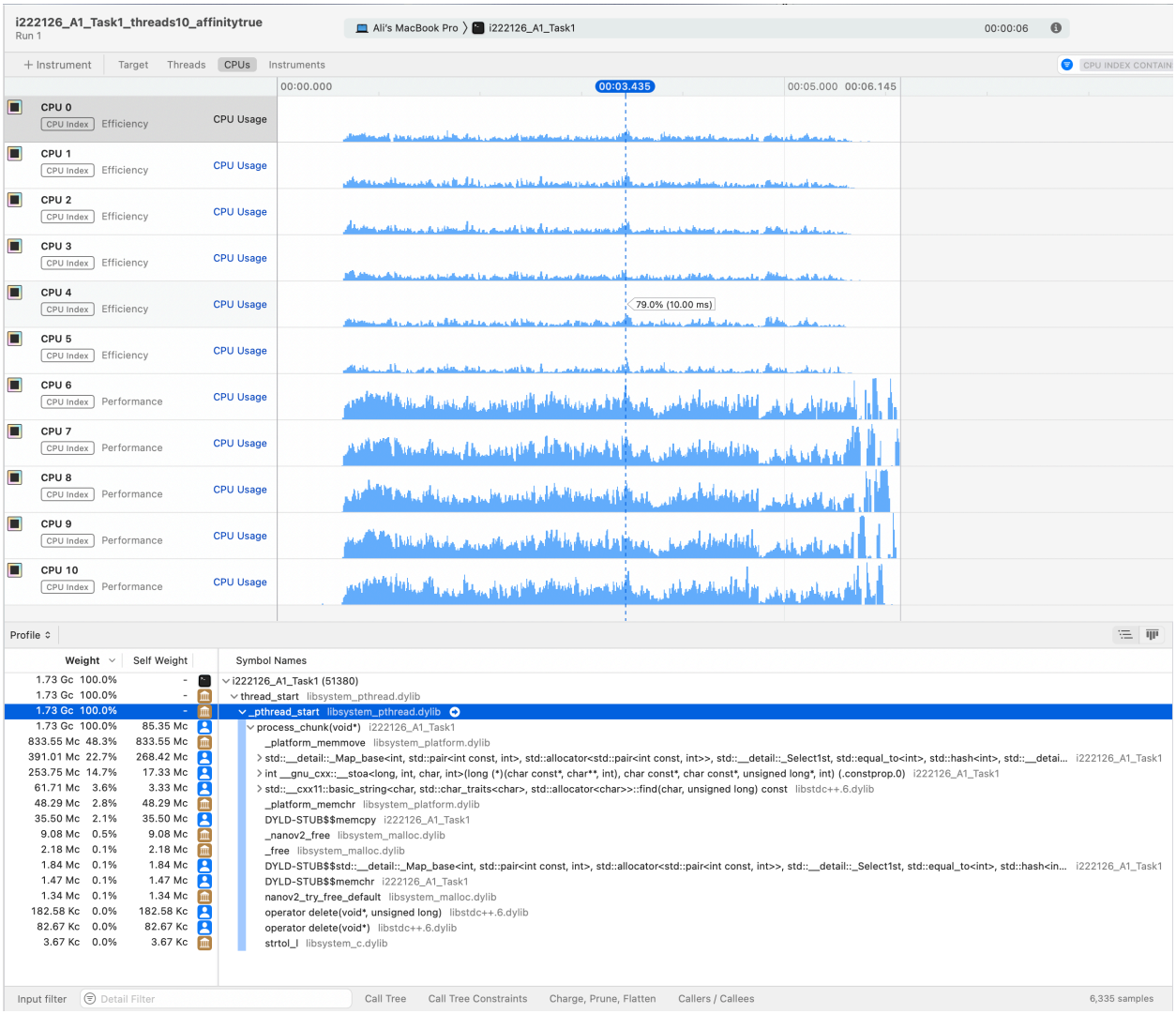
Results

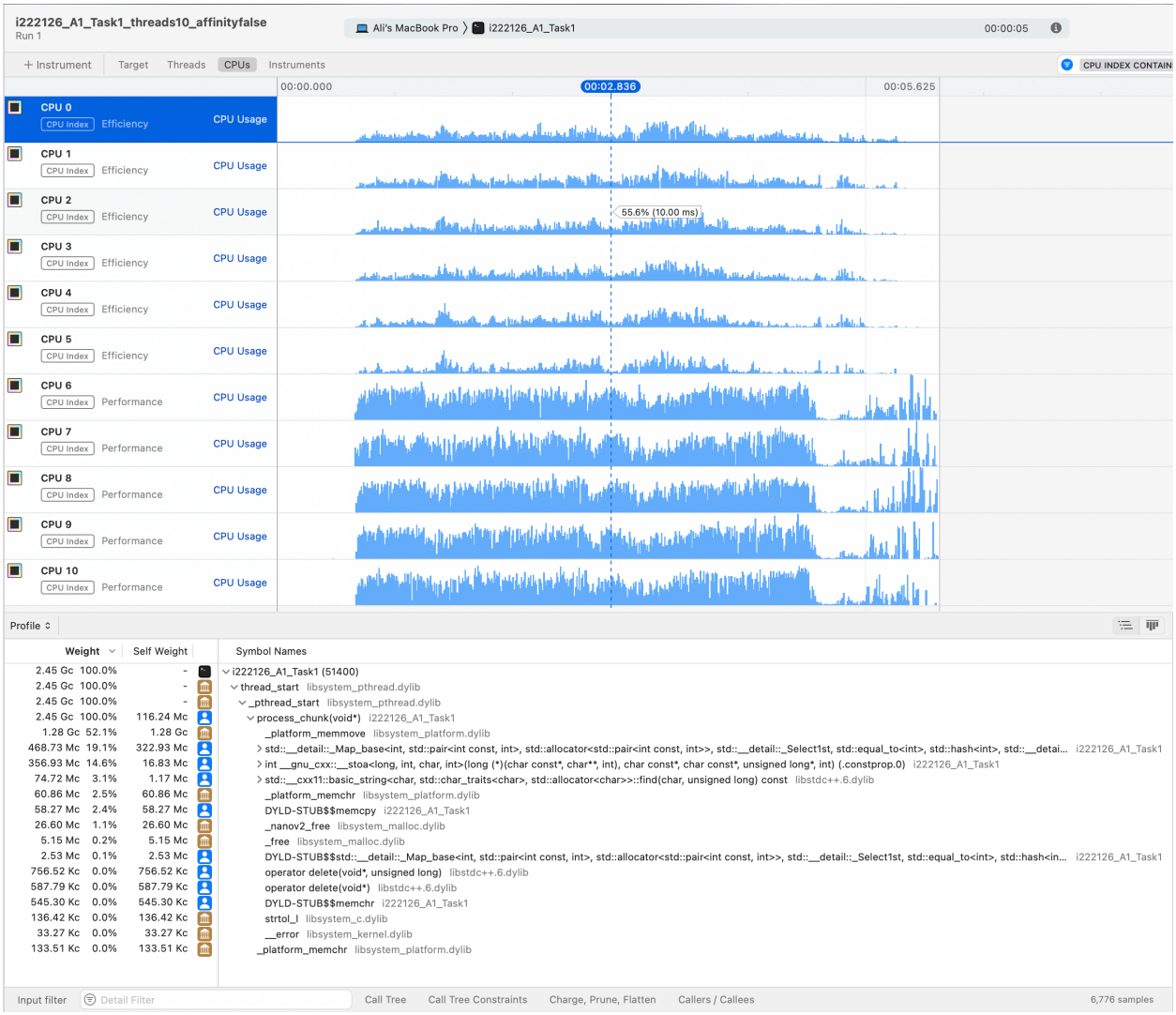


The performance on macOS is nearly identical with and without thread affinity. The execution times follow the same trend, with minimal variation. Enabling thread affinity doesn't provide a significant advantage for this workload. The performance improves with thread count, but beyond 12-16 threads, execution time stabilizes and slightly increases due to overheads.

This suggests macOS's scheduler efficiently distributes threads, making affinity settings less impactful compared to other operating systems where affinity control yields more noticeable improvements.

CPU Utilization





Observations:

- Thread affinity enabled (top image): Execution time is 6.145 seconds, CPU usage is evenly distributed among cores, workload distribution is structured, total sample weight is 1.73 Gc.
- Thread affinity disabled (bottom image): Execution time is 5.625 seconds, CPU usage is well-distributed but with more variation, total sample weight is 2.45 Gc, more CPU cycles consumed, as threads shift between cores, CPU allocation is less predictable.

Key differences:

- Performance improvement: Without affinity, execution time is reduced, suggesting thread migration might benefit load balancing.
- CPU usage variation: Affinity enforces thread pinning to specific cores, leading to a structured workload, while affinity disabled causes more variability.
- More samples recorded: Affinity disabled takes more samples (2.45 Gc vs 1.73 Gc), indicating more system activity due to context switching.

Hotspots:

- process_chunk(void) in consumes the significant portion of execution time.
- Memory operations in libsystem_platform.dylib and libsystem_malloc.dylib suggest potential memory bottlenecks.
- String processing functions indicate potential optimization targets for searching operations within strings.

Recommendations:

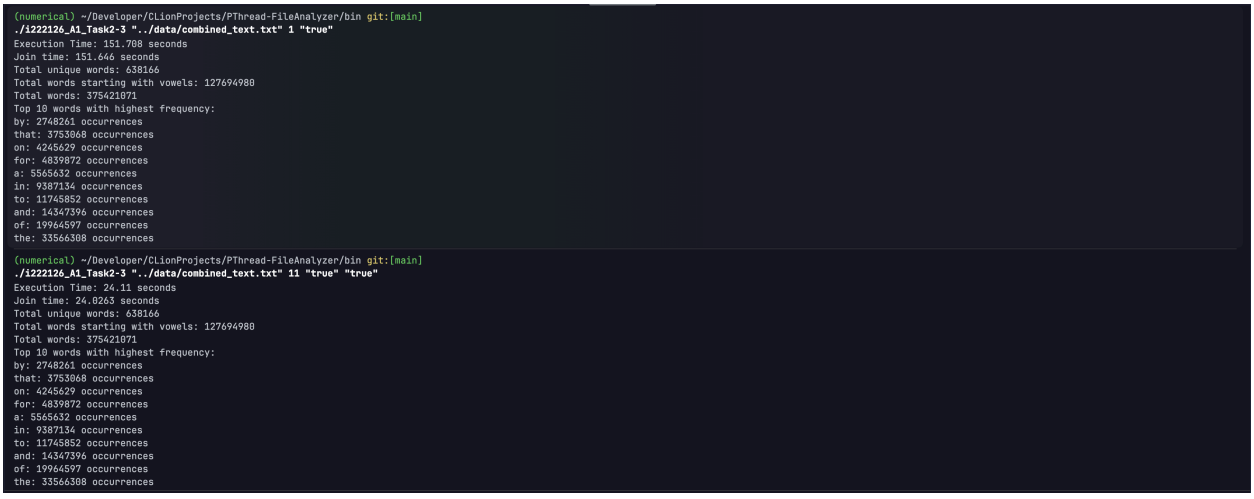
- Disabling affinity enforcement for faster execution.

Task 2-3 : Text File Analysis

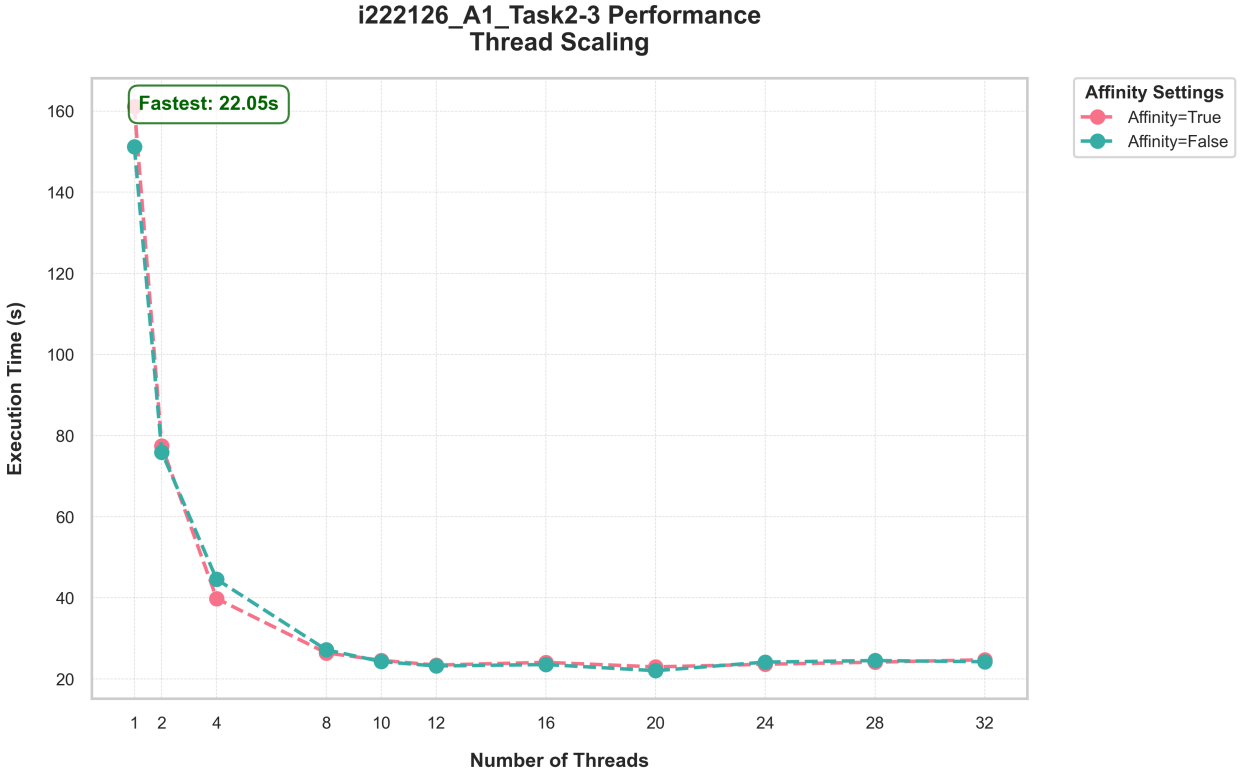
Approach

After memory-mapping the file and calculating chunk offsets, the main function assigns chunks to individual threads. Each thread processes its chunk by iterating through the lines, extracting words (filtering non-alphabetic characters and converting them to lowercase), and counting total words, words beginning with vowels, and their frequencies. Threads update a shared data structure in a thread-safe manner using mutex locks. Once all threads finish (ensured by joining), the main thread aggregates the results—calculating overall statistics and the top 10 most frequent words via a min-heap—and optionally saves the outcome to a file before cleaning up and exiting.

Sample Output

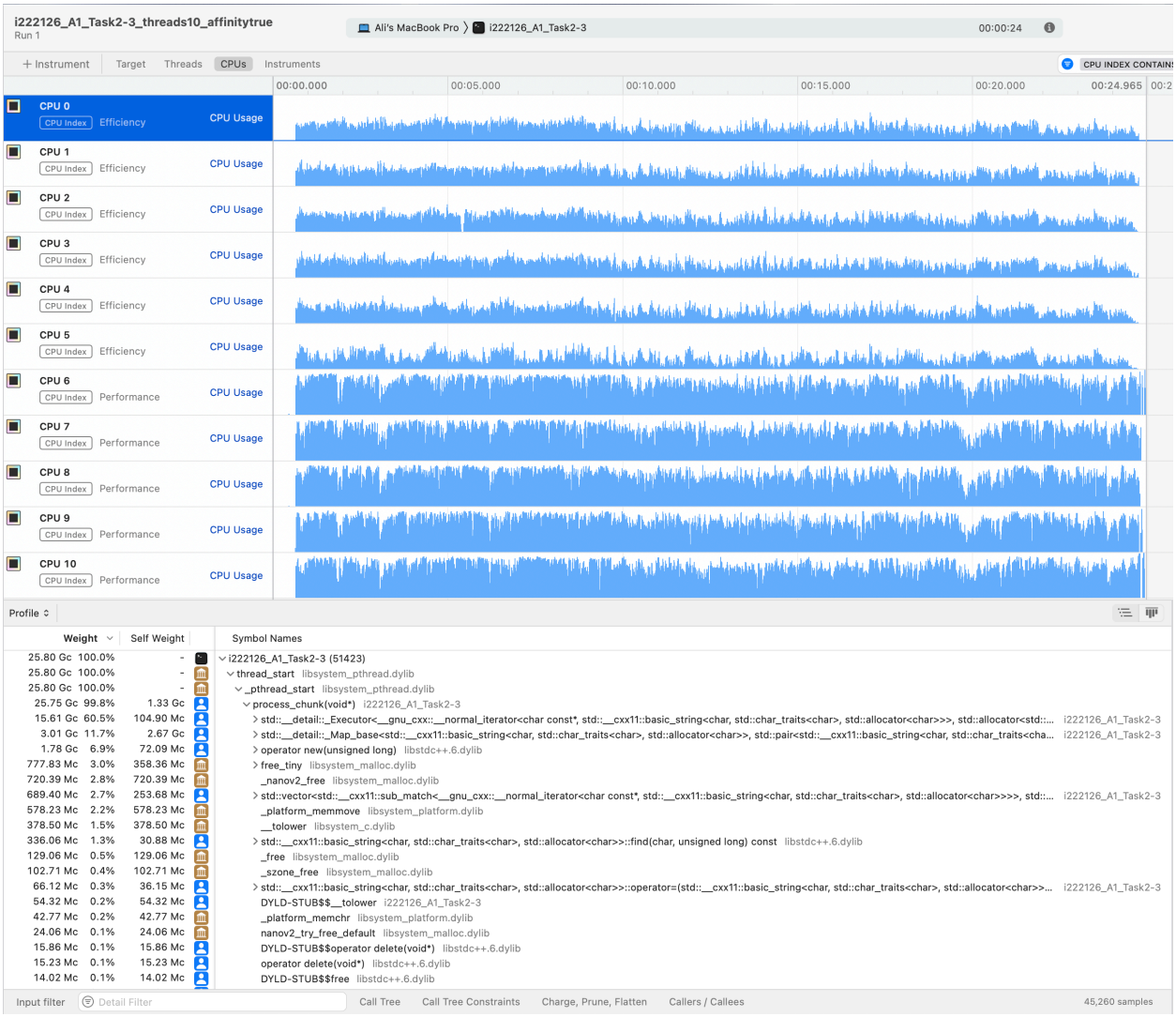


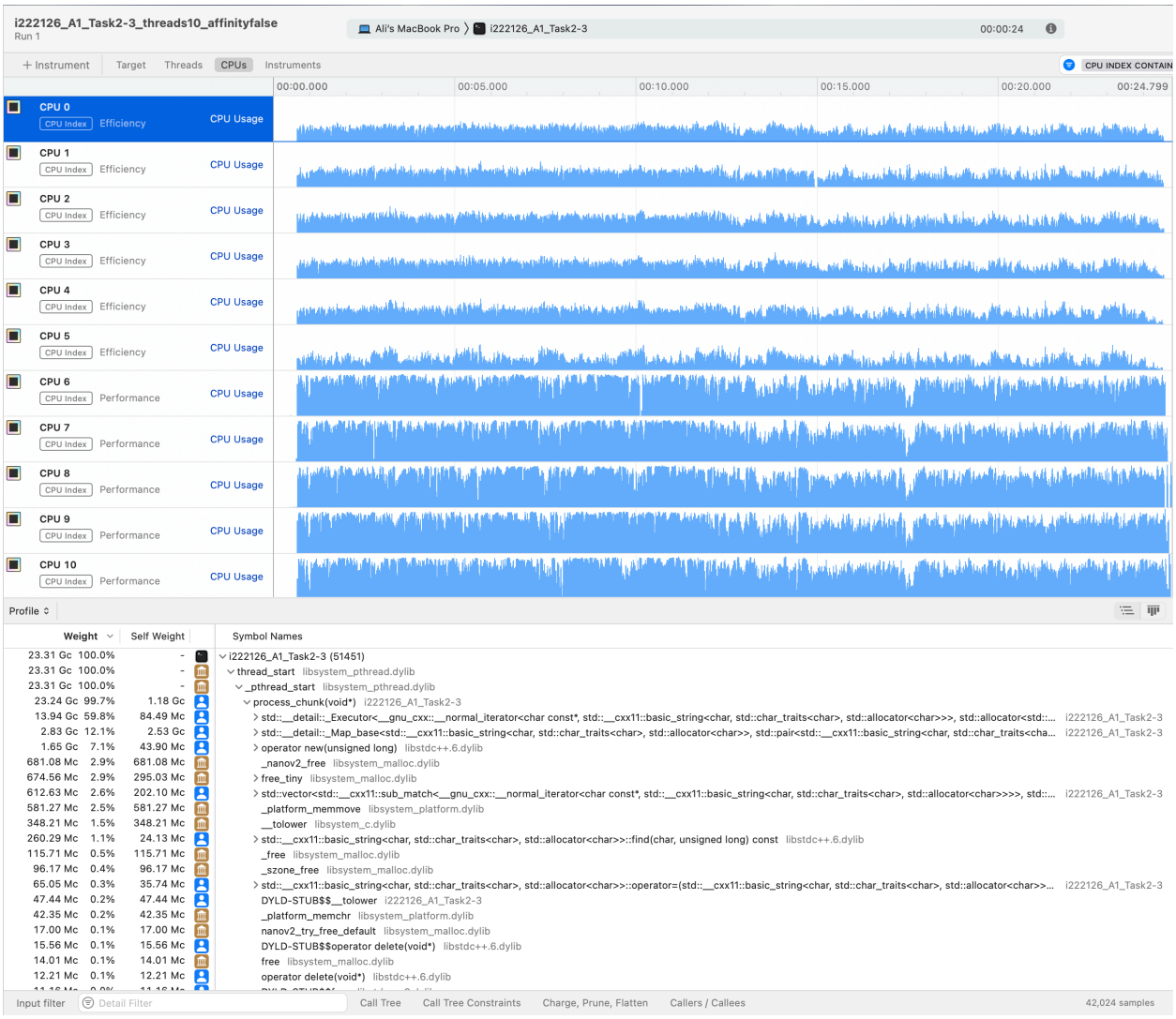
Results



Similar to the results of Task 1, performance improves with thread count, but beyond 12-16 threads, execution time stabilizes and slightly increases due to overheads. This implies parallel processing benefits up to a point, after which additional threads don't contribute further gains and may introduce inefficiencies. Thread affinity settings don't significantly improve performance, suggesting either the scheduler overrides manual affinity or the workload doesn't benefit from such control, unlike other operating systems.

CPU Utilization





Observations:

- Thread affinity enabled (top image): Execution time is 24.965 seconds, CPU usage is evenly distributed among cores, workload distribution is structured, total sample weight is 25.80 Gc.
- Thread affinity disabled (bottom image): Execution time is 24.799 seconds, CPU usage is well-distributed but with similar variation, total sample weight is 23.31 Gc.

Key differences:

- Performance improvement: Without affinity, execution time is reduced, suggesting MacOS scheduler is better equipped to handle thread assignment to CPU cores
- CPU usage variation: Affinity enforces thread pinning to specific cores, leading to a structured workload, while affinity disabled causes more variability.
- Less cycles without affinity further support the idea that MacOS scheduler is more efficient and effective at assigning threads to cores than the programmer.

Hotspots:

- process_chunk(void) in consumes the significant portion of execution time.
- Operations in string Map and iterating over string are using the most CPU.

Recommendations:

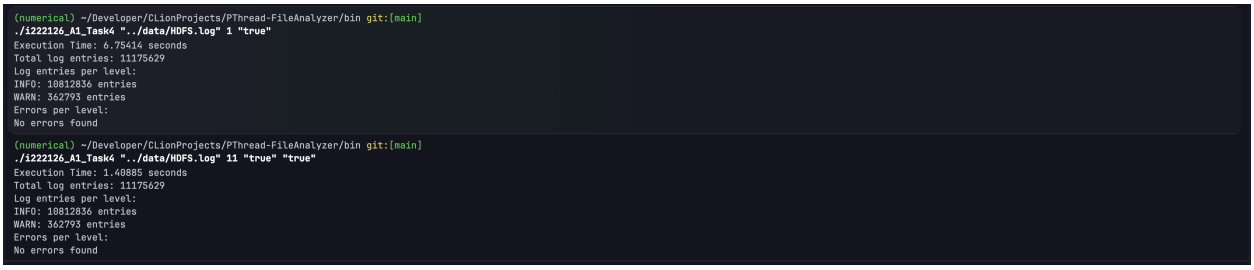
- Disabling affinity enforcement for faster execution. Although in this case there is little difference.

Task 4 : Log File Analysis

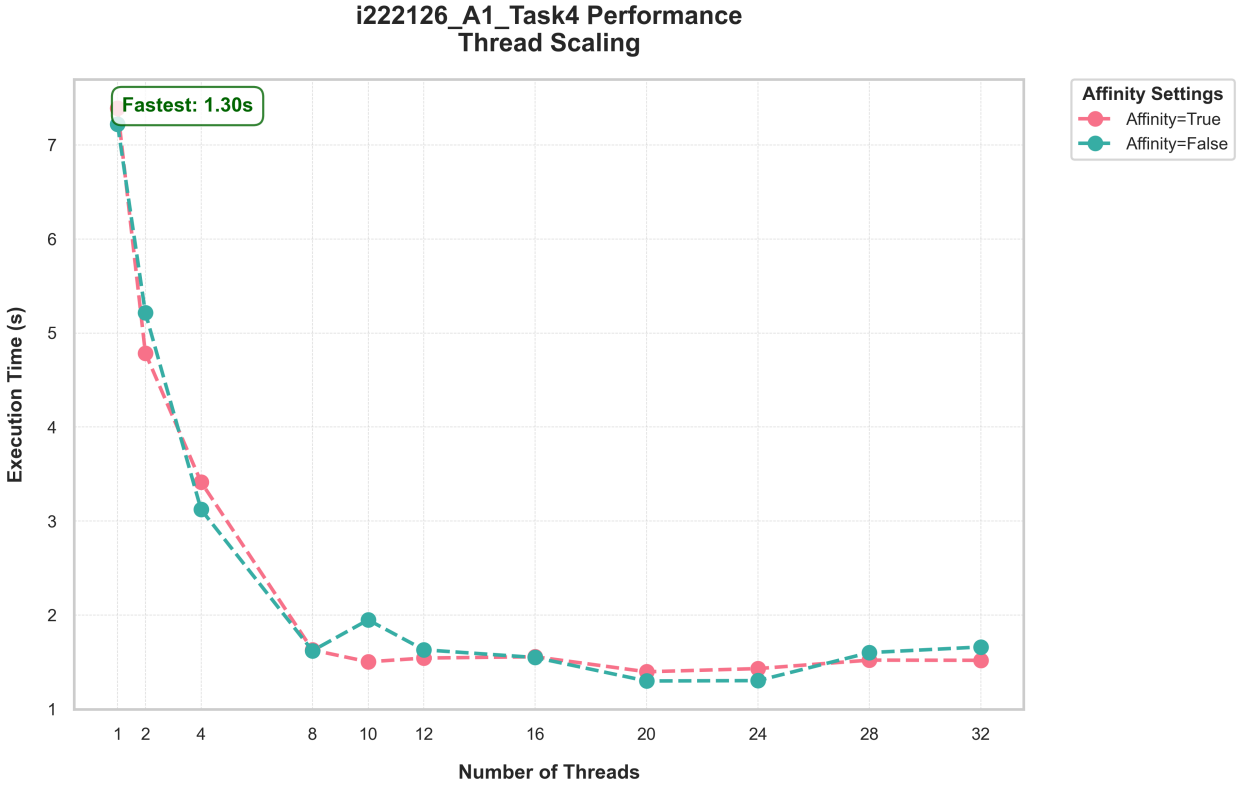
Approach

After memory mapping the file and calculating chunk offsets, the main function assigns chunks to individual threads. Each thread processes its chunk independently, parsing log lines to extract details such as date, time, log level, and message components (e.g., block, source, destination). It counts log entries, categorizes them by log level, and tracks errors per level and unique logs. Once all threads complete, the main function aggregates the results, including total log entries, logs per level, and errors per level. This approach ensures parallel processing of log data, with each thread handling a portion of the file, and the main thread consolidating the results for output.

Sample Output



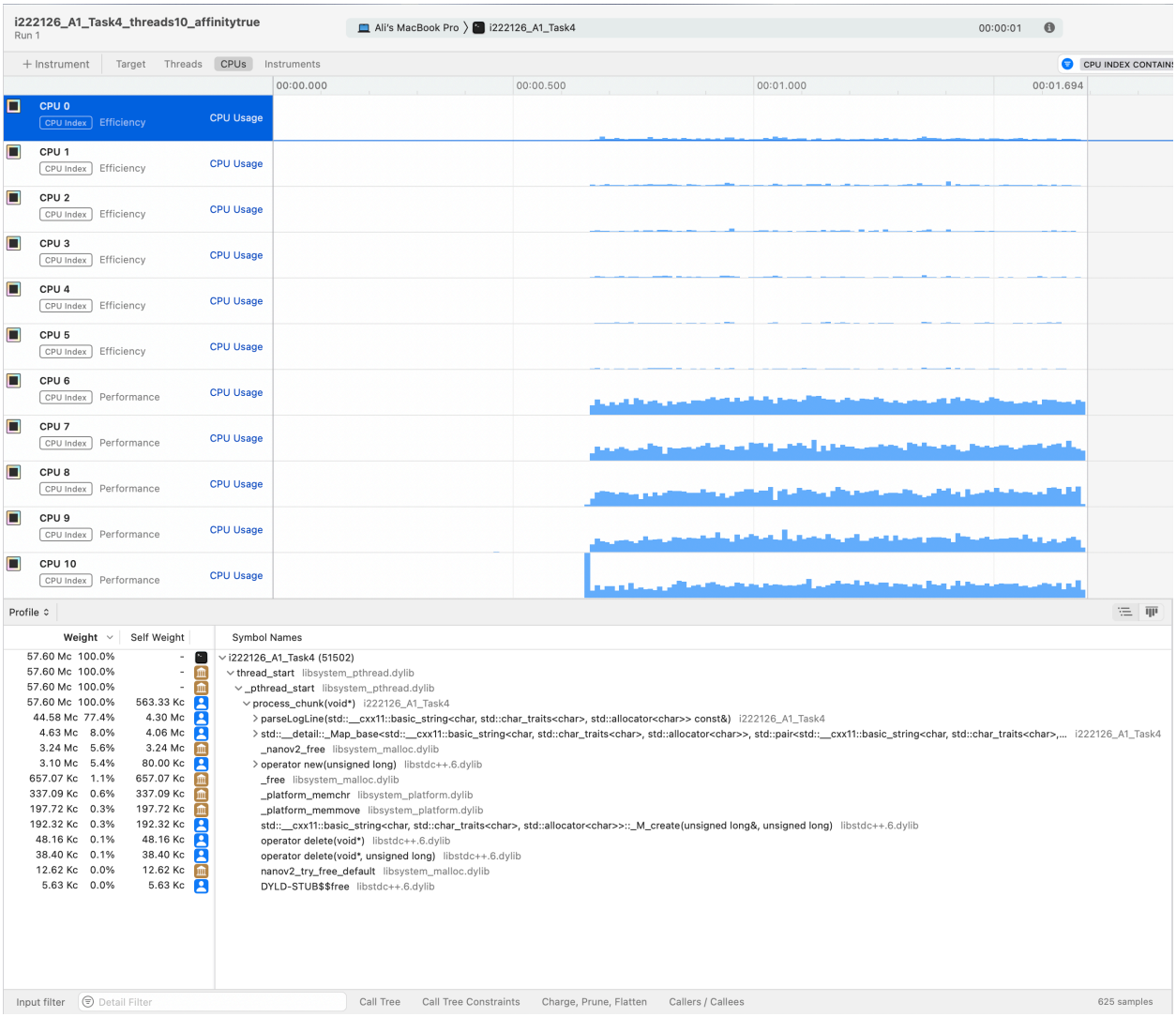
Results

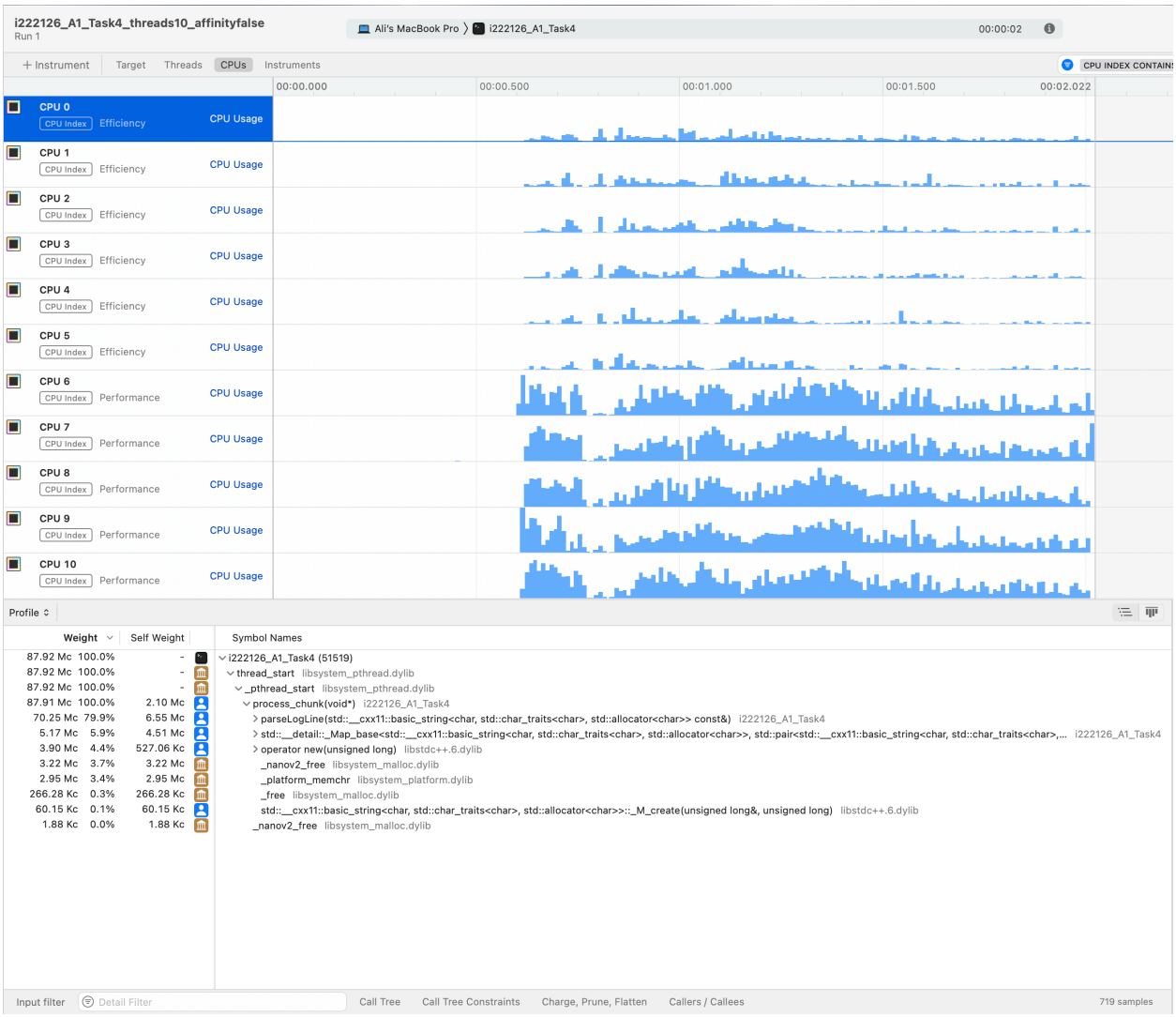


In this experiment, the best performance is achieved with 20 and 24 threads, unlike previous findings where performance stabilized or degraded beyond 12-16 threads. This suggests that the task benefits from a higher number of threads due to its nature or workload distribution.

Despite the deviation, thread affinity has minimal impact, as execution times are nearly identical. The system's scheduler efficiently manages thread distribution at higher thread counts. Overall, the optimal thread count varies depending on the workload, with higher counts (20 and 24) yielding the best performance.

CPU Utilization





Observations:

- Thread affinity enabled (top image): Execution time is 1.694 seconds, CPU usage is evenly distributed among cores, workload distribution is structured, total sample weight is 57.60 Mc.
- Thread affinity disabled (bottom image): Execution time is 2.022 seconds, CPU usage is well-distributed but with more variation, total sample weight is 87.92 Mc, more CPU cycles consumed, as threads shift between cores, CPU allocation is less predictable.

Key differences:

- In this case with Affinity execution time is less but as execution time is short it may also be due to noise from other processes utilizing the CPU.
- CPU usage variation: Affinity enforces thread pinning to specific cores, leading to a structured workload, while affinity disabled causes more variability.
- More samples recorded: Affinity disabled takes more samples (87.92 Mc vs 57.60 Mc), indicating more system activity due to context switching.

Hotspots:

- process_chunk(void) in consumes the significant portion of execution time.
- The function parseLogLine is reading and processing log lines, which likely involves frequent string manipulations, thus showing high utilization.
- Maps with strings as key introduces more overhead sue to frequent lookups and insertions.

Recommendations:

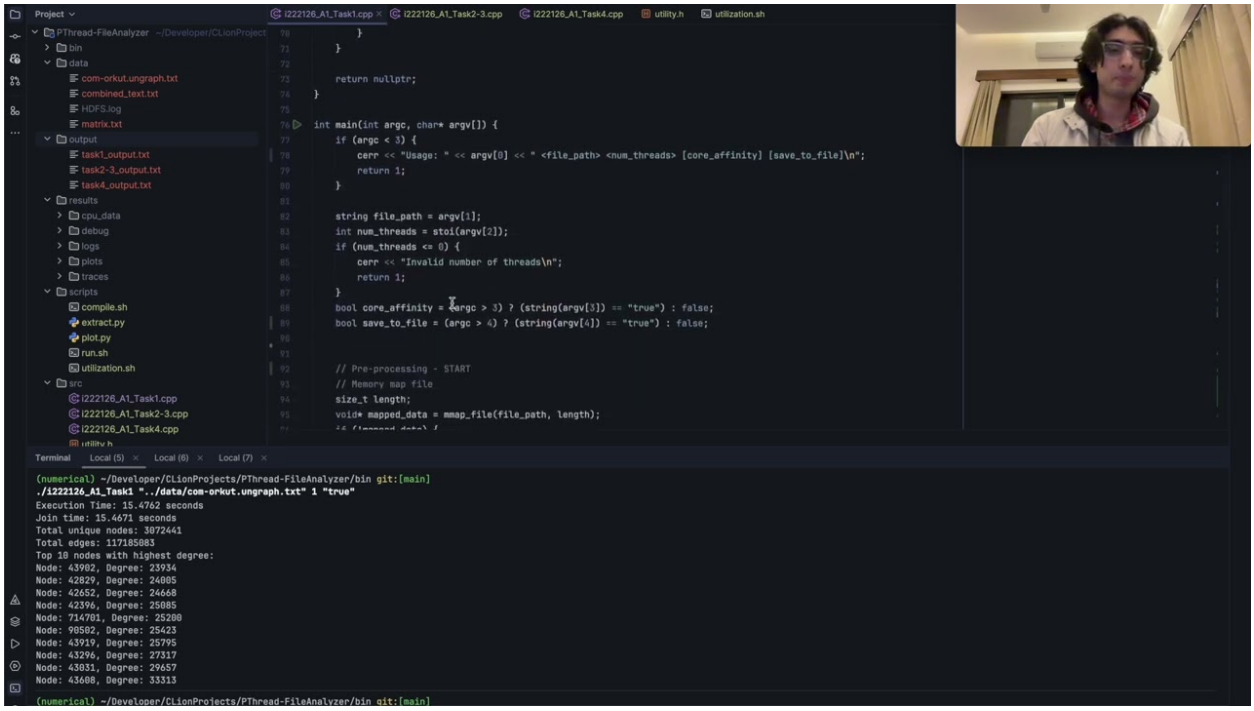
- Attempt to improve and minimize string operations.

Conclusion

The assignment required applying multi-threaded approaches to efficiently process large text files. By dividing the file into chunks and assigning each to a separate thread, the provided implementation optimizes extraction and analysis. Thread synchronization techniques ensure data integrity, while optional CPU affinity settings enhance performance on multi-core systems. The final results, including word statistics and frequency rankings, demonstrate the effectiveness of this approach in handling large-scale file processing tasks. Overall, this method significantly improves processing speed compared to a single-threaded implementation, making it well-suited for high-performance file analysis.

Links

Video



Datasets

- Task 1 : com-orkut.ungraph.txt.gz
- Task 2-3 : English portion of MultiUN
- Task 4 : HDFS_v1